

MATEMATIČKI FAKULTET, UNIVERZITET U BEOGRADU

RAČUNARSKA INTELIGENCIJA

SEMINARSKI RAD

Optimizacija raspoređivanja poslova na mašinama primenom metaheurističkih algoritama

(Minimum Job Shop Scheduling)

Mentor:

Stefan Kapunac

Katedra za računarstvo i
informatiku

Student:

Nenad Pešić 62/2022

Beograd, 2026.

Sadržaj

1	Uvod	3
1.1	Opis problema	3
1.2	Složenost	3
1.3	Pregled literature	5
1.3.1	Domaći izvori	5
1.4	Sadržaj i doprinos rada	5
2	Opis rešenja	7
2.1	Kodiranje rešenja	7
2.1.1	Prirodan zapis i njegov nedostatak	7
2.1.2	Permutacija sa ponavljanjem	8
2.1.3	Višeznačnost zapisa	8
2.2	Dekoder	8
2.2.1	Postupak	9
2.2.2	Poluaktivni rasporedi	9
2.2.3	Složenost	9
2.2.4	Primer	9
2.3	Kritični put	10
2.3.1	Definicija	10
2.3.2	Značaj	10
2.3.3	Određivanje	11
2.3.4	Primer	11
2.4	Donja granica	12
2.4.1	Dva trivijalna ograničenja	12
2.4.2	Ograničenja pristupa	12
2.4.3	Dokaz optimalnosti bez rešavača	13
2.5	Iscrpna pretraga	13
2.5.1	Postupak	13
2.5.2	Granica primenljivosti	14
2.5.3	Uloga u radu	14
2.6	Okoline	14
2.6.1	Zamena dva elementa	14
2.6.2	Okolina po kritičnom putu	15

2.6.3	Poređenje okolina	15
2.7	Lokalna pretraga	16
2.7.1	Postupak	16
2.7.2	Prvo naspram najboljeg poboljšanja	16
2.7.3	Višestruko pokretanje	16
2.8	Simulirano kaljenje	17
2.8.1	Kriterijum prihvatanja	17
2.8.2	Postupak	17
2.9	Metoda promenljivih okolina	18
2.9.1	Niz okolina	18
2.9.2	Postupak	18
2.10	Genetski algoritam	19
2.10.1	Postupak	19
2.10.2	Selekcija	19
2.10.3	Ukrštanje	19
2.10.4	Mutacija i elitizam	20
2.11	Celobrojni linearni model	21
2.11.1	Promenljive	21
2.11.2	Ograničenja	21
2.11.3	Veličina modela	21
3	Eksperimentalni rezultati	23
3.1	Okruženje za eksperimentisanje	23
3.2	Test instance	23
3.3	Metodologija	24
3.3.1	Budžet umesto vremena	24
3.3.2	Broj ponavljanja	25
3.3.3	Stohastičnost i reproducibilnost	25
3.4	Rezultati testiranja	26
3.5	Egzaktno rešavanje	27
3.6	Poređenje sa rezultatima iz literature	29
3.7	Diskusija	30
3.7.1	Uža okolina ne donosi bolji rezultat	30
3.7.2	Složenija metoda nije nužno bolja	32
3.7.3	Odnos prema egzaktnom rešavanju	32
4	Zaključak	33
5	Literatura	34

Poglavlje 1

Uvod

1.1 Opis problema

Problem raspoređivanja poslova po mašinama (engl. *job shop scheduling problem*) jedan je od najviše proučavanih problema kombinatorne optimizacije. Postavka je sledeća.

Zadato je n poslova i na raspolaganju je m mašina (resursa). Svaki pojedinačni posao J_j sastoji se od niza operacija koje moraju biti izvršene **tačno zadatim redosledom**. Pored toga, svaka operacija o izvršava se na specifičnoj mašini $M(o)$ i traje p_o jedinica vremena. Traži se raspored poslova sa ******najmanjim ukupnim trajanjem. Pri tome, moraju se poštovati sledeća ograničenja:

1. **Redosled unutar posla mora biti očuvan.** Operacija o_k može započeti izvršavanje tek kada je gotovo izvršavanje operacija $o_1 \dots o_{k-1}$.
2. **Najviše jedna operacija može da se izvršava u jednom trenutku na jednoj mašini.** Da bi operacija o započela izvršavanje, mašina $M(o)$ mora biti slobodna.

Ukupno trajanje jednog ovakvog rasporeda nazivamo *makespan* i označavamo sa C_{max} .

Na slici iznad jasno možemo videti razliku između lošeg i dobrog rasporeda. Za instancu koja sadrži svega dva posla i dve mašine prvi raspored traje 7, a drugi čak 11 jedinica vremena.

Uticaj optimizacije još je приметniji na većoj instanci **ft06**

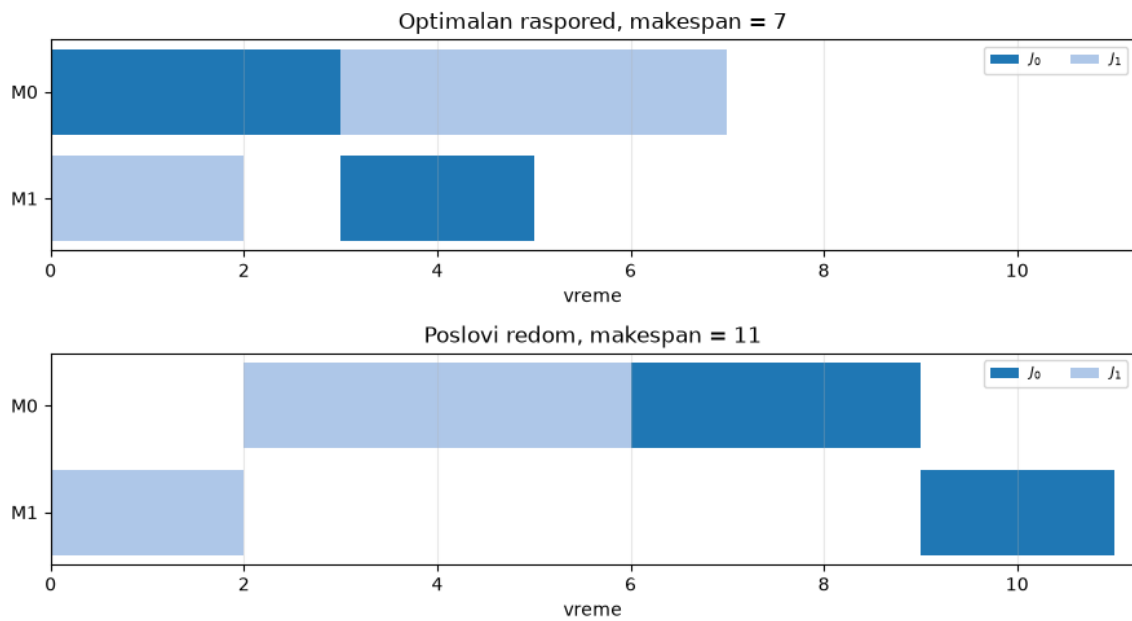
1.2 Složenost

Već pri tri mašine problem postaje NP-težak (Garey, Johnson, i Sethi 1976), a u opštem slučaju svrstava se u najteže probleme raspoređivanja zbog svojih specifičnosti. (Garey, Johnson, i Sethi 1976)

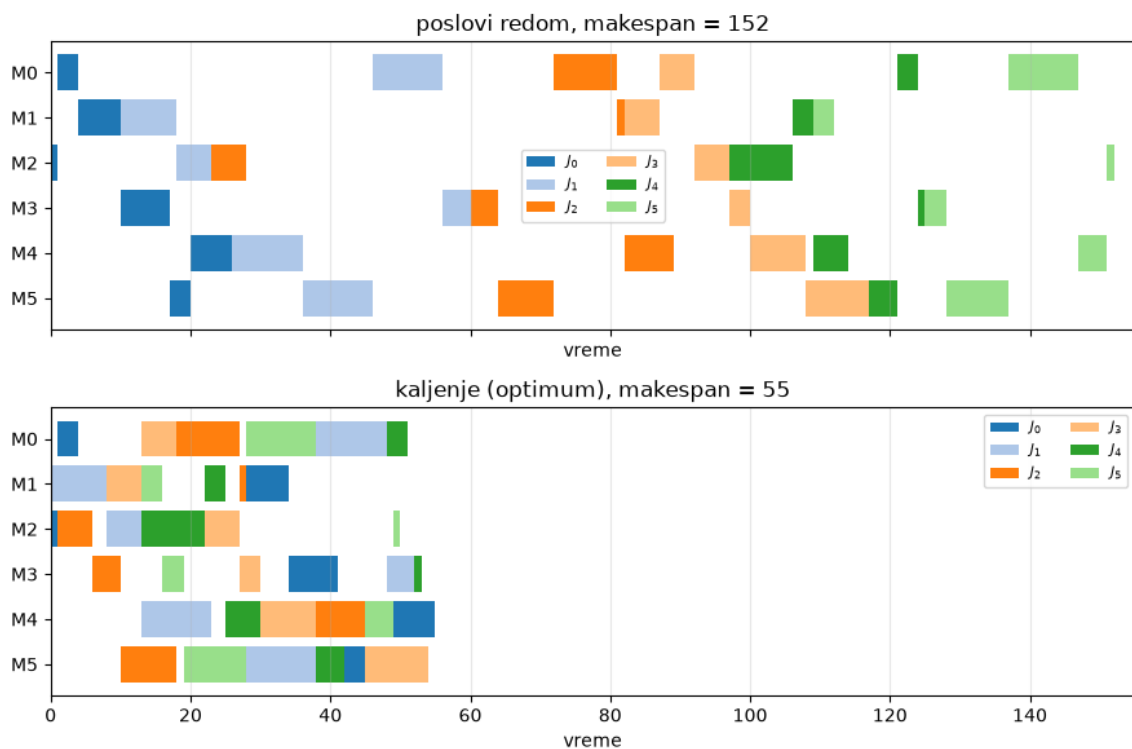
Ovu tvrdnju dodatno utvrđuje veličina prostora pretrage. Naime, za svaku mašinu biramo redosled operacija koje se na njoj izvršavaju, pa je broj kombinacija reda $(n!)^m$. Kod instance sa šest poslova i šest mašina dobijamo procenu od oko $1,4 \cdot 10^{17}$ kombinacija, pri čemu mnoge od tih kombinacija ne predstavljaju validan raspored.

Problem se u praksi pokazao toliko težak da je instanca **ft10**, sa svojih deset poslova i deset mašina, koja je objavljena 1963. godine (Fisher i Thompson 1963), uprkos *skromnim* zahtevima rešena tek **dvadeset šest godina kasnije**¹.

¹Optimalno rešenje ove konkretne instance dokazali su tek Džek Karlijer i Erik Pinson 1989. godine (Carlier i



Slika 1.1: Dva rasporeda iste instance sa dva posla i dve mašine, prikazana u istoj razmeri. Gornji raspored traje 7, donji 11 vremenskih jedinica. Operacije i njihova trajanja su u oba slučaja identični — razlikuje se samo redosled kojim zauzimaju mašine.



Slika 1.2: Dva rasporeda instance **ft06** sa šest poslova i šest mašina. Gornji nastaje izvršavanjem poslova jednog za drugim i traje 152 vremenske jedinice, donji je optimalno rešenje sa trajanjem 55.

1.3 Pregled literature

Prve formulacije problema kao zadatka celobrojnog linearnog programiranja dali su Vagner (Vagner 1959) i Mane (Manne 1960). Maneova, disjunktivna formulacija iskorišćena je i u ovom radu za formiranje egzaktnog rešenja. Metode granjanja i ograđivanja dovele su do dokazivanja optimuma instance `ft10` (Carlier i Pinson 1989), a potom i do sistematskog rešavanja standardnih zbirki (Applegate i Cook 1991).

Gifler i Thompson (Giffler i Thompson 1960) uveli su pojam aktivnog rasporeda i pokazali da taj skup sadrži optimalno rešenje. Na toj strukturi, a pre svega na pojmu kritičnog puta, kasnije su građene okoline kod heurističkih metoda. Van Larhoven sa saradnicima (Laarhoven, Aarts, i Lenstra 1992) spojili su takvu okolinu sa simuliranim kaljenjem, a Novicki i Smutnicki (Nowicki i Smutnicki 1996) sa tabu pretragom. Za genetske algoritme, Birvirt (Bierwirth 1995) je predložio kodiranje permutacijom sa ponavljanjem koje je korišćeno i ovde, dok su u (Bierwirth, Mattfeld, i Kopfer 1996) upoređeni operatori ukrštanja prilagođeni takvom zapisu. Pregled razvoja oblasti dat je u (Jain i Meeran 1999).

Test instance korišćene u radu potiču od Fišera i Tompsona (Fisher i Thompson 1963) i Lorensa (Lawrence 1984), a dostupne su preko zbirke OR-Library (Beasley 1990).

1.3.1 Domaći izvori

Na razumevanje samih metaheuristika, njihove strukture i međusobnih odnosa, najviše je uticala domaća literatura. Materijali sa vežbi iz predmeta Računarska inteligencija (Kapunac 2026) pomogli su u konceptualnom razumevanju implementiranih metaheuristika i njihovu praktičnu primenu. Udžbenik *Veštačka inteligencija* (Janičić i Nikolić 2025) poslužio je kao dodatni izvor pri razumevanju genetskih algoritama i njegove primene.

Strana literatura navedena u prethodnom odeljku korišćena je pre svega za deo koji je specifičan za sam problem raspoređivanja. Prvenstveno za kodiranje rešenja, strukturu rasporeda, konstrukciju okolina i celobrojni formulaciju, ali i za poređenje dobijenih rezultata sa objavljenim optimalnim vrednostima.

1.4 Sadržaj i doprinos rada

U ovom radu problem je formulisan i rešavan koristeći dva pristupa. Najpre je formulisano zajedničko kodiranje rešenja i zajednički dekodir. Zatim je nad tom formulacijom problem rešen putem četiri metaheuristike:

1. Lokalna pretraga sa ponovnim pokretanjem
2. Simulirano kaljenje
3. Metoda promenljivih okolina
4. Genetski algoritam

Nakon toga, problem je formulisan i kao zadatak celobrojnog linearnog programiranja (engl. *ILP*) i rešen egzaktnim rešavačem, radi utvrđivanja stvarnih optimuma i nezavisne provere ispravnosti implementacije.

Sve metode upoređene su pod istim uslovima uloženog truda. Dodeljen im je jednak budžet

Pinson 1989).

izražen brojem poziva funkcije cilja (dekodiranja), pokretane su trideset nezavisnih puta nad devet test instanci različitih dimenzija.

Rad je organizovan na sledeći način. Poglavlje 2 opisuje kodiranje rešenja, dekodir, pomoćne strukture i sve razvijene metode. Poglavlje 3 sadrži eksperimentalne rezultate, poređenje sa vrednostima iz literature i raspravu o zapaženim pojavama. Poglavlje 4 daje zaključak i moguće pravce daljeg rada.

Poglavlje 2

Opis rešenja

2.1 Kodiranje rešenja

Za početak potrebno je formalno definisati rešenje ka kojem stremimo i odrediti format pogodan za primenu optimizacija koje slede. U ovom poglavlju obrađeno je upravo to pitanje, ali i sve pomoćne strukture neophodne za uspešno implementiranje optimizacija. Dati su opisi oblika okoline, načina ukrštanja, načina evaluacije i drugi.

2.1.1 Prirodan zapis i njegov nedostatak

Redosled operacija unutar posla zadat je instancom i ne može se menjati. Jedina dimenzija slobode koju rešavač poseduje jeste određivanje **redosleda koji operacije zauzimaju na svakoj mašini**. Nameće se, dakle, prirodan zapis, kao m permutacija, po jedan raspored za svaku mašinu.

Takav zapis ima ozbiljan nedostatak koji se vrlo brzo prikazuje. Naime, većina kombinacija ne odgovara nijednom validnom rasporedu. Posmatrajmo instancu sa dva posla i dve mašine:

$$J_1 : M_1(3) \rightarrow M_2(2), \quad J_2 : M_2(2) \rightarrow M_1(4)$$

Mašina M_1 obrađuje prvu operaciju posla J_1 i drugu operaciju posla J_2 , dok mašina M_2 obrađuje prvu operaciju posla J_2 i drugu operaciju posla J_1 . Za svaku mašinu postoje dva moguća redosleda, dakle ukupno četiri kombinacije:

Tabela 2.1: Sve kombinacije redosleda po mašinama za instancu sa dva posla i dve mašine.

redosled na M_1	redosled na M_2	C_{max}
J_1 pa J_2	J_2 pa J_1	7
J_1 pa J_2	J_1 pa J_2	11
J_2 pa J_1	J_2 pa J_1	11
J_2 pa J_1	J_1 pa J_2	—

Poslednja kombinacija **ne opisuje validan raspored**. Prva operacija posla J_1 čeka da se na mašini M_1 završi druga operacija posla J_2 . Ta operacija čeka prvu operaciju istog posla. Dalje, sad ona čeka da se na mašini M_2 završi druga operacija posla J_1 , a ta operacija čeka prvu

operaciju posla J_1 , od koje smo i pošli. Zavisnosti čine ciklus i nijedna operacija ne može da počne.

Na ovako maloj instanci neizvodljiva je jedna od četiri kombinacije. Sa porastom broja poslova i mašina taj udeo raste, pa bi svaka metoda koja slučajno bira redoslede po mašinama najveći deo rada trošila na odbacivanje neispravnih rešenja.

2.1.2 Permutacija sa ponavljanjem

Zbog toga je usvojeno kodiranje koje je za ovaj problem predložio Kristijan Birvirt (Bierwirth 1995). Rešenje je **lista oznaka poslova**, u kome se oznaka svakog posla javlja onoliko puta koliko taj posao ima operacija. Za instancu sa dva posla po dve operacije, jedan takav niz je

$$(J_2, J_1, J_1, J_2).$$

Niz se čita sleva nadesno, a k -to pojavljivanje oznake posla J_j predstavlja **k -tu operaciju** tog posla. Gornji niz stoga redom označava prvu operaciju posla J_2 , prvu i drugu operaciju posla J_1 , pa drugu operaciju posla J_2 .

Ključna osobina ovog zapisa jeste da je **svaki niz sa ispravnim brojem pojavljivanja izvodljiv**. $(k+1)$. operacija nekog posla ne može se pojaviti pre k -te, jer je njeno mesto u nizu po definiciji kasnije pojavljivanje iste oznake. Redosled unutar posla time nije ograničenje koje se proverava, već osobina samog zapisa, pa ciklus opisan u prethodnom odeljku nije moguće ni zapisati u Birvirtovoj notaciji.

Broj različitih nizova za instancu sa poslovima $J_1, \dots, J_n$ iznosi

$$\frac{(\sum_j |J_j|)!}{\prod_j |J_j|!},$$

što za instancu **ft06** sa 36 operacija daje približno $2,67 \cdot 10^{24}$ različitih nizova.

2.1.3 Višeznačnost zapisa

Jedan od izazova ove notacije jeste to što ne garantuje uzajamnu jednoznačnost zapisa i rasporeda. Odnosno, više različitih nizova može dati isti raspored. Za pomenutu instancu sa dva posla postoji šest različitih nizova, koji se preslikavaju u svega tri izvodljiva rasporeda iz tabele 2.1. Čak četiri od šest nizova daju optimalno rešenje.

Prostor pretrage je, dakle, veći od prostora rešenja. Ipak, u literaturi je cena koja se plaća za garantovanu izvodljivost smatrana prihvatljivom. Navodi se (a što nije ovim radom eksplicitno proveravano) da je provera izvodljivosti u alternativnom zapisu znatno skuplja od višestrukog obilaska istog rasporeda.

2.2 Dekoder

Glavna mana Birvirtove notacije jeste što niz oznaka ne nosi eksplicitno informaciju o rasporedu poslova na mašini. Ona jeste sadržana unutar njega, ali je potrebno niz adekvatno transformisati

ba bismo do nje došli. Postupak kojim od niza dolazimo do rasporeda, odnosno svakoj operaciji dodeljuje trenutak početka, nazivamo *dekoderom*.

2.2.1 Postupak

Operacije se obrađuju redom kojim se javljaju u nizu. Za svaku od njih trenutak početka određen je sa dva ograničenja: operacija ne može početi pre nego što se završi prethodna operacija istog posla, niti pre nego što se oslobodi mašina koju zahteva. Stoga, najraniji trenutak koji zadovoljava oba jeste

$$s_o = \max(r_{J(o)}, f_{M(o)}),$$

gde je $r_{J(o)}$ trenutak završetka prethodne operacije posla kome o pripada, a $f_{M(o)}$ trenutak u kome se oslobađa mašina $M(o)$. Nakon dodele, obe vrednosti ažuriraju se na $s_o + p_o$.

Dekoder time održava tri niza vrednosti: vreme spremnosti svakog posla, vreme oslobađanja svake mašine i redni broj sledeće operacije svakog posla (poput brojača instrukcija u savremenim računarima). Poslednji od njih pretvara oznaku posla u konkretnu operaciju, u skladu sa pravilom o k -tom pojavljivanju.

Vrednost funkcije cilja, C_{max} , jednaka je najvećem trenutku završetka među svim operacijama.

2.2.2 Poluaktivni rasporedi

Ovako opisan postupak svaku operaciju smešta u **najraniji mogući trenutak** pri zadatom redosledu. Rasporedi sa tom osobinom nazivaju se *poluaktivnim* i nose osobinu da se nijedna operacija ne može pomeriti ulevo bez promene redosleda na nekoj mašini.

Poluaktivni rasporedi nisu najuži skup koji vredi razmatrati. Uži je skup *aktivnih* rasporeda, u kome se nijedna operacija ne može pomeriti ulevo ni uz promenu redosleda. Poznato je da taj skup sadrži bar jedno optimalno rešenje (Giffler i Thompson 1960). Postupak Giffler–Thompsona proizvodi upravo takve rasporede i time smanjuje prostor pretrage.

U ovom radu korišćen je poluaktivni dekokder, iz dva razloga. Najpre zato što je poluaktivni dekokder izuzetno jednostavan za implementaciju, a pritom skup poluaktivnih rasporeda i dalje sadrži optimalno rešenje. Jedina cena jednostavnosti je veći prostor pretrage.

2.2.3 Složenost

Dekoder obavlja jedan prolaz kroz niz i za svaku operaciju konstantan broj radnji, pa je njegova složenost $O(L)$, gde je $L = \sum_j |J_j|$ ukupan broj operacija.

Kako je dekokder jedino mesto na kome se rešenje vrednuje, broj njegovih poziva korišćen je kao mera uloženog rada pri poređenju metoda, na način opisan u odeljku o metodologiji.

2.2.4 Primer

Za instancu iz odeljka o kodiranju i niz (J_1, J_2, J_1, J_2) dekokder redom dodeljuje:

Tabela 2.2: Rad dekodera nad nizom (J_1, J_2, J_1, J_2) . Vrednost C_{max} jednaka je 7.

korak	operacija	mašina	spreman posao	slobodna mašina	početak	kraj
1	J_1 , prva	M_1	0	0	0	3
2	J_2 , prva	M_2	0	0	0	2
3	J_1 , druga	M_2	3	2	3	5
4	J_2 , druga	M_1	2	3	3	7

U trećem koraku jasno možemo videti prvo ograničenje na delu. Operacija čeka da se završi prva operacija istog posla iako je mašina slobodna. U četvrtom koraku ograničenje nameće mašina. Bitno je uočiti tu razliku, s obzirom da nam je potrebna prilikom konstrukcije kritičnog puta, čiji postupak je opisan u narednoj sekciji.

2.3 Kritični put

U prethodnom odeljku opisali smo postupak koji nam iz Birvirtovog zapisa rasporeda poslova daje vrednost C_{max} . Ipak, vrednost C_{max} govori o tome koliko raspored traje, ali ne nosi informaciju **šta ga određuje**. Za dalju analizu, a pre svega za konstrukciju okolina, potrebno je znati koje operacije zaista utiču na dužinu rasporeda, a koje imaju vremenskog prostora.

2.3.1 Definicija

Kritični put je niz operacija $o_1, o_2, \dots, o_k$ takav da važi

$$s_{o_1} = 0, \quad f_{o_i} = s_{o_{i+1}}, \quad f_{o_k} = C_{max},$$

dakle lanac koji počinje u trenutku nula, završava se u trenutku C_{max} i u kome između uzastopnih operacija nema praznog hoda. Svaki par uzastopnih operacija povezan je jednim od dva ranije navedena ograničenja. Posao ne može da počne ili zbog toga što čeka na prethodnu operaciju istog posla ili na oslobađanje potrebne mašine.

Iz uslova da praznog hoda nema neposredno sledi

$$C_{max} = \sum_{i=1}^k p_{o_i},$$

odnosno dužina kritičnog puta jednaka je vrednosti funkcije cilja.

2.3.2 Značaj

Neposredna posledica prethodne jednakosti jeste da se C_{max} može smanjiti **isključivo** promenom koja utiče na neku od operacija sa kritičnog puta. Operacije van puta nemaju uticaj na ukupnu dužinu puta i mogu se proizvoljno pomerati dokle god ostaju van kritičnog puta.¹

Ovo nosi značajan uvid pri formiranju okoline neke tačke. Naime, smisleno je razmatrati samo one okoline koje menjaju kritični put. Taj postupak opisan je u odeljku o okolinama.

¹Vredi napomenuti da je operacije van kritičnog puta moguće izmestiti tako da *produže* ukupno vreme trajanja i time pogoršaju ukupni rezultat. Ipak, u tom slučaju bi se i one same našle na kritičnom putu.

2.3.3 Određivanje

Određivanje kritičnog puta moguće je učiniti blagom izmenom funkcije dekodera. Pri dodeli trenutka početka,

$$s_o = \max(r_{J(o)}, f_{M(o)}),$$

jedan od dva argumenta je veći i on je razlog zašto operacija nije mogla ranije. Ako je veći bio $r_{J(o)}$, operaciju je zadržala prethodna operacija istog posla. U suprotnom, ako je bio $f_{M(o)}$, zadržala ju je prethodna operacija na istoj mašini. Pamćenjem te odluke (npr. u matrici) dobija se, za svaku operaciju, pokazivač na njenog **neposrednog prethodnika**.

Kritični put se zatim dobija polazeći od operacije sa najvećim trenutkom završetka i praćenjem pokazivača unazad, sve do operacije koja počinje u trenutku nula. Postupak je složenosti $O(L)$, kao i sam dekodir.

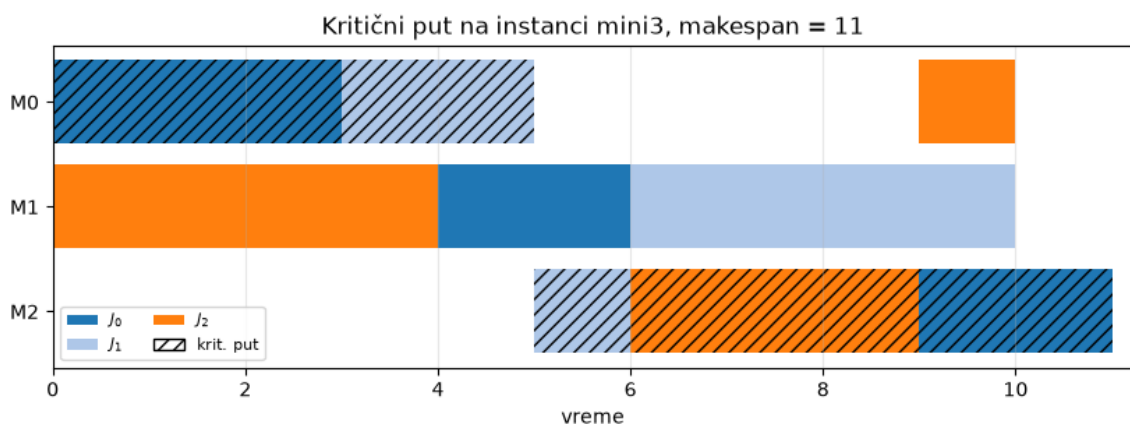
Radi efikasnosti, pamćenje prethodnika izdvojeno je u zasebnu funkciju. Osnovni dekodir, koji se poziva pri svakom vrednovanju, ne obavlja taj posao, dok se prošireni poziva samo kada je kritični put zaista potreban.

2.3.4 Primer

Za instancu `mini3` i niz koji daje optimalno rešenje, kritični put čini pet od devet operacija:

Tabela 2.3: Kritični put instance `mini3` pri optimalnom rasporedu. Zbir trajanja iznosi 11, što je jednako vrednosti C_{max} .

operacija	mašina	trajanje	početak	kraj	zadržao je
J_1 , prva	M_1	3	0	3	—
J_2 , prva	M_1	2	3	5	mašina
J_2 , druga	M_3	1	5	6	posao
J_3 , druga	M_3	3	6	9	mašina
J_1 , treća	M_3	2	9	11	mašina



Slika 2.1: Raspored dobijen dekodiranjem niza $(J_1, J_2, J_2, J_3, J_1, J_2, J_3, J_1, J_3)$ nad instancom `mini3`. Šrafirane su operacije koje pripadaju kritičnom putu.

Primetimo da na slici 2.1 treća operacija J_2 može biti pomerena za jednu jedinicu vremena unapred bez uticaja na optimalnost rešenja. Slično, operacije koje se izvršavaju na mašini M_2 mogu biti u potpunosti ispremeštane sve dok poštuju uslov dovršavanja prethodnih instrukcija svojih poslova.

2.4 Donja granica

Vrednost koju metoda pronađe sama po sebi ne daje nam informaciju o kvalitetu datog rešenja. Kako bismo mogli to da utvrdimo potrebno je da imamo informaciju o dokazanom optimumu (ukoliko je instanca problema formalno rešiva²) sa kojim bismo uporedili ovaj rezultat. Druga mogućnost jeste da nekako izračunamo donju granicu rešenja, odnosno vrednost za koju možemo da tvrdimo da je manja od ili jednaka optimalnom rešenju. Što veću takvu vrednost nađemo to nam je procena verodostojnija i korisnija u analizi metoda optimizacije.

2.4.1 Dva trivijalna ograničenja

Prvo ograničenje nameće posao. Operacije jednog posla izvršavaju se jedna za drugom, pa od početka prve do kraja poslednje protekne bar zbir njihovih trajanja. Kako to važi za svaki posao, važi i za najduži među njima:

$$C_{max} \geq \max_j \sum_{o \in J_j} p_o.$$

Drugo ograničenje nameće mašina. Mašina obrađuje jednu operaciju u datom trenutku, pa se sve operacije dodeljene jednoj mašini izvršavaju uzastopno. Čak i kada mašina nikada ne stoji, poslednja operacija završava se najranije u trenutku koji je jednak zbiru trajanja svih operacija na toj mašini:

$$C_{max} \geq \max_k \sum_{o \in M_k} p_o.$$

Obe nejednakosti važe za svaki validan raspored, pa važi i veća od njih:

$$LB = \max \left(\max_j \sum_{o \in J_j} p_o, \max_k \sum_{o \in M_k} p_o \right).$$

Izračunavanje zahteva jedan prolaz kroz instancu, dakle složenosti je $O(L)$, pri čemu se dekodir ne poziva nijednom.

2.4.2 Ograničenja pristupa

Ovako dobijena granica zanemaruje **čekanje** (prazan hod). Prvi argument pretpostavlja da posao nikada ne čeka slobodnu mašinu, drugi da mašina nikada ne stoji prazna. U stvarnim rasporedima dešava se i jedno i drugo, pa je granica po pravilu strogo manja od optimuma.

Koliko je manja, zavisi od instance:

²U razumnom vremenu

Tabela 2.4: Odstupanje donje granice od poznatog optimuma, poredano po veličini odstupanja.

instanca	donja granica	optimum	odstupanje
1a01	666	666	0
1a05	593	593	0
1a03	588	597	1,5 %
1a02	635	655	3,1 %
ft20	1119	1165	3,9 %
1a04	537	590	9,0 %
mini3	10	11	9,1 %
ft06	47	55	14,5 %
ft10	655	930	29,6 %

Na instanci **ft10** granica je za skoro trećinu ispod optimuma i tu je praktično beskorisna. Na **1a01** i **1a05**, međutim, poklapa se sa optimumom.

2.4.3 Dokaz optimalnosti bez rešavača

Donja granica nam pored procene optimalnosti može dati i čvrst dokaz iste u određenim situacijama. Naime, ukoliko na instanci I za koju važi donja granica LB , pronađemo raspored čija je vrednost $C_{max} = LB$ onda smo sigurni da je takav raspored optimalan. Po definiciji donje granice nijedan raspored ne može biti bolji od nje, pa je onaj čija je vrednost upravo ona sigurno najbolji mogući.

Upravo zbog ove osobine dostizanje vrednosti 666 na instanci **1a01**, odnosno 593 na **1a05** dokazuje optimalnost istog.

2.5 Iscrpna pretraga

Pre implementiranja bilo koje heuristike korisno nam je da imamo postupak koji garantuje optimalno rešenje, makar i po cenu izrazito velikog (nekada nedostižnog) vremena izvršavanja. Takav postupak može nam služiti kao sredstvo validacije. Na primerima gde ga je moguće izvršiti on će nam dati informaciju o optimalnom rezultatu, a tom informacijom možemo da evaluiramo rešenja dobijena primenom neke od heuristika.

2.5.1 Postupak

Iscrpna pretraga (metoda grube sile) prolazi kroz **sve različite nizove** iz odeljka o kodiranju, dekodira svaki i pamti najbolji. Kako je svaki niz izvodljiv, nikakva provera ispravnosti nije potrebna, a kako se svaki raspored može zapisati bar jednim nizom, optimum se sigurno nalazi među razmotrenim rešenjima.

Nizovi se nabrajaju kao permutacije multiskupa, dakle bez ponavljanja istih rasporeda oznaka. Za instancu sa poslovima $J_1, \dots, J_n$ njihov broj iznosi

$$\frac{(\sum_j |J_j|)!}{\prod_j |J_j|!}.$$

Nizovi se obrađuju jedan po jedan, bez čuvanja u memoriji, pa je prostorna složenost zanemarljiva. Vremenska složenost jednaka je proizvodu gornjeg izraza i cene jednog dekodiranja (u našem slučaju $O(L)$ gde je L dužina niza).

2.5.2 Granica primenljivosti

Broj nizova raste brže od eksponencijalne funkcije, pa je pretraga upotrebljiva samo na vrlo malim instancama. Za instance sa tri mašine i rastućim brojem poslova dobija se:

Tabela 2.5: Rast prostora pretrage sa brojem poslova, pri tri mašine.

poslova	operacija	broj nizova	procenjeno vreme
3	9	1 680	trenutno
4	12	369 600	oko 1 s
5	15	168 168 000	oko 10 min
6	18	$1,3 \cdot 10^{11}$	oko 5 dana

Već pri šest poslova pretraga prestaje da bude izvodljiva. Za instancu **ft06**, koja sadrži svega šest poslova i šest mašina, broj različitih nizova iznosi približno $2,67 \cdot 10^{24}$; pri milion dekodiranja u sekundi obilazak bi trajao oko $8,5 \cdot 10^{10}$ godina, dakle višestruko duže od procenjene starosti svemira.

2.5.3 Uloga u radu

Iscrpna pretraga primenjena je na instancu **mini3**, sa tri posla i tri mašine, kao i na dve još manje instance korišćene pri razvoju. Dobijeni optimum od 11 za **mini3** jedina je vrednost u ovom radu koja je **dokazana neposredno**, obilaskom celog prostora rešenja³.

Ta vrednost korišćena je kao provera ispravnosti dekodera i svih razvijenih metoda. Zaista, metoda koja na **mini3** ne dostiže 11 sadrži grešku, jer nijedna heuristika ne sme biti lošija od tačnog odgovora na instanci ove veličine.

2.6 Okoline

Svaka od metoda razvijenih u ovom radu (izuzev genetskog algoritma), prostor rešenja pretražuje počevši od dostavljene početne tačke i istražujući njenu neposrednu okolinu, tražeći (možda lokalno) optimalno rešenje. Zbog toga neophodno je definisati *okolinu* određenog Birvirtovog niza na koju će se osloniti sve metaheuristike.

2.6.1 Zamena dva elementa

Najjednostavnija okolina rasporeda u Birvirtovom zapisu može se dobiti **zamenom mesta** dve-ma operacijama u nizu. S obzirom da rezultat ovakve transformacije daje permutaciju početnog niza znamo da je taj raspored sigurno validan.

³Kasnije će biti reči o rešavanju egzaktnim metodama putem rešavača CPLEX. Bez obzira na to, ovo predstavlja značajan rezultat i oslonac za ostatak rada imajući u vidu jednostavnost implementacije i sigurnost u dobijeno rešenje

Zamena dve iste oznake ne menja niz, pa se takvi parovi preskaču. Za niz dužine L broj suseda je stoga najviše $\binom{L}{2}$, a u praksi manji za broj parova jednakih oznaka. Orijentacije radi, na instanci **ft06** to daje 540 suseda, a na **ft10** i **ft20** po 4500, odnosno 4750.

Ovakva definicija okoline ne koristi nikakvo znanje o problemu. Ne služi se osobinom o kritičnom putu koju smo naveli ranije u radu i predstavlja skup permutacija dobijenih jednom zamenom. To je ujedno i glavna mana ovakve definicije okoline. Naravno, prednost se ogleda u jednostavnosti implementacije i ceni jednog generisanja koje se izvršava u konstantnom vremenu.

2.6.2 Okolina po kritičnom putu

Iz osobine kritičnog puta, izvedene ranije, sledi da izmena koja ne dira nijednu operaciju sa puta ne može smanjiti C_{max} . Prirodno je stoga razmatrati samo izmene koje ga dodiruju.

Upravo tu osobinu koristi N_1 okolina koju su predložili Piter van Larhoven i saradnici (Laarhoven, Aarts, i Lenstra 1992). Ona obuhvata zamene **susednih operacija na kritičnom putu koje se izvršavaju na istoj mašini**. Oba uslova su neophodna za uspešnost definicije ove okoline. Najpre susednost obezbeđuje da izmena utiče na kritičan put, a zajednička mašina obezbeđuje da je izmena validna (jer se redosled unutar posla ne sme menjati).

Sužavanje prostora pretrage je znatno:

Tabela 2.6: Prosečan broj suseda po koraku, izmeren nad 200 slučajno izabranih nizova.

instanca	zamena dva elementa	N_1	odnos
ft06	540	6,7	80,6×
ft10	4500	17,4	259,3×
ft20	4750	31,6	150,4×

2.6.3 Poređenje okolina

Očekivano bi bilo da uža okolina, koja razmatra samo poteze sa izgledom na uspeh, daje bolje rezultate. Merenje pokazuje suprotno. U tabeli su prikazani rezultati pri izjednačenom budžetu od 200 000 poziva dekodera, kroz 10 pokretanja:

Tabela 2.7: Prosečna vrednost funkcije cilja pri izjednačenom budžetu, po okolini.

instanca	metoda	zamena	N_1
ft06	lokalna pretraga	55,0	55,0
ft06	kaljenje	55,0	55,0
ft10	lokalna pretraga	1050,5	1063,6
ft10	kaljenje	956,7	1001,0
ft20	lokalna pretraga	1305,1	1431,2
ft20	kaljenje	1182,1	1223,7

Uzroci su razmotreni u odeljku o diskusiji rezultata.

Zbog ovakvih rezultata je u svim metodama razvijenim u ovom radu korišćena okolina zamene dva elementa, dok je N_1 implementirana i izmerena, ali nije ušla u konačan izbor.

2.7 Lokalna pretraga

Najjednostavnija metaheuristička pretraga jeste upravo lokalna pretraga. Ideja je jednostavna: pretragu započinjemo iz početne tačke (recimo nasumično zadat raspored), a zatim se krećemo u pravcu najboljeg suseda sve dok takav postupak donosi poboljšanje.

2.7.1 Postupak

ulaz: *instanca*, početni niz *s0*

izlaz: lokalni optimum

```
v <- dekodiraj(s)
ponavljaj
  najbolji <- nijedan
  za svakog suseda t iz okoline(s):
    w <- dekodiraj(t)
    ako je w < v:
      v <- w
      najbolji <- t
  ako najbolji ne postoji: stani
  s <- najbolji
vрати (v, s)
```

Postupak staje kada nijedan sused nije bolji od tekućeg rešenja. Dobijeno rešenje je *lokalni optimum* u odnosu na izabranu okolinu.

2.7.2 Prvo naspram najboljeg poboljšanja

Goreprikazana varijanta pretražuje celu okolinu trenutne tačke a zatim bira najboljeg suseda (*najbolje poboljšanje*). Postoje i druge varijante lokalne pretrage. Jedna od njih predstavlja modifikaciju datog algoritma tako da se trenutni korak prekida kod prvog boljeg suseda, čime je korak jeftiniji ali manje precizan.

Na izabranim test primerima pokazalo se da daju slična rešenja, dok je metoda *prvog poboljšanja* značajno jeftinija⁴. Varijanta najboljeg rešenja korišćena je prilikom nezavisnog testiranja lokalne pretrage, a u implementaciji metode promenljivih okolina korišćena je varijanta prvog poboljšanja.

2.7.3 Višestruko pokretanje

S obzirom da je potrebno normalizovati heuristike po nekoj zajedničkoj skali kako bismo mogli da uporedimo njihove performanse pri uloženom istom trudu i resursima, potrebno je bilo i lokalnu pretragu prilagoditi tom uslovu. U opisanom obliku ona radi dok ne dođe do nekog optimuma u čijoj okolini nema pogodnije tačke što iziskuje različit trud u zavisnosti od početnog stanja.

Metoda je prilagođena tako da pokreće više različitih instanci dok ne premaši dati budžet, što se najbolje može videti u kodu koji sledi.

⁴U odnosu na broj pozivanja funkcije dekodiranja

```

najbolje <- inf
dok budzet nije potrosen:
  s <- slucajna permutacija
  (v, s) <- lokalna_pretraga(istanca, s)
  ako je v < najbolje:
    najbolje <- v

```

2.8 Simulirano kaljenje

Simulirano kaljenje (Kirkpatrick, Gelatt, i Vecchi 1983) nadograđuje lokalnu pretragu tako što se **ponekad pomera u smeru pogoršanja**, nadajući se da će ga to odvesti ka optimalnijem rešenju u budućnosti. Verovatnoća prihvatanja zavisi od stepena pogoršanja i parametra T , koji se smanjuje tokom pretrage.

2.8.1 Kriterijum prihvatanja

Ako prelaz sa tekućeg rešenja na suseda menja vrednost funkcije cilja za $\Delta = w - v$, prelaz se prihvata sa verovatnoćom

$$p = \begin{cases} 1, & \Delta < 0 \\ e^{-\Delta/T}, & \Delta \geq 0 \end{cases}$$

Izraz je poznat kao *Metropolisov kriterijum* (Metropolis i dr. 1953). Dakle verovatnoća da ćemo prihvatiti lošije rešenje eksponencijalno opada pri većem stepenu pogoršanja kako pretraga ne bi otišla predaleko od poželjnog rešenja.

Takođe, na početku pretrage, kada parametar T ima visoku vrednost pretraga prihvata gotovo svaki prelaz, dok se usmerava ka sve boljem rešenju kako se budžet iscrpljuje.

2.8.2 Postupak

ulaz: istanca, početni niz s, budžet B
 izlaz: najbolje viđeno rešenje

```

# inicijalizacija
tekuće <- s
v <- dekodiraj(s)
najbolje <- v
najbolji_niz <- s
T <- T0
alpha <- (Tk / T0)^(1/B)

dok budzet nije potrosen:
  t <- nasumicna tacka iz okoline(tekuće)
  w <- dekodiraj(t)
  delta <- w - v

```

```

    ako je delta < 0 ili slučajan_broj() < e^(-delta/T):
        tekuće <- t
        v <- w
        ako je w < najbolje:
            najbolje <- w
            najbolji_niz <- t
    T <- T * alpha
    vrati (najbolje, najbolji_niz)

```

Vredno je naglasiti da je potrebno **posebno pamtit**i najbolje viđeno rešenje. Glavni razlog jeste upravo u konstrukciji samog algoritma pretrage, tj. u činjenici da prihvatamo i prelaske u lošija stanja, čime se gubi garancija da je poslednji potez zaista video najbolje rešenje.

2.9 Metoda promenljivih okolina

Metoda promenljivih okolina (Mladenović i Hansen 1997) preduzima drugačiju strategiju kako bi umakla lokalnim optimumima. Naime, koristi se koncept **protresanja**, koji podrazumeva inkrementalno pomeranje od trenutnog rešenja za sve veću udaljenost sve dok to pomeranje ne donese poboljšanje.

2.9.1 Niz okolina

Definiše se niz okolina $N_1 \subset N_2 \subset \dots \subset N_{k_{max}}$, gde N_k obuhvata nizove koji se od tekućeg razlikuju za k uzastopnih zamena. Veće k znači veću udaljenost od tekućeg rešenja.

2.9.2 Postupak

ulaz: instanca, početni niz s, budžet B, kmax
 izlaz: najbolje viđeno rešenje

```

    tekuće <- s
    najbolje <- dekodiraj(s)

    dok budžet nije potrošen:
        k <- 1
        dok je k < kmax i budžet nije potrošen:
            t <- protresi(tekuće, k)
            (w, t') <- lokalna_pretraga(instanca, t)
            ako je w < najbolje:
                najbolje <- w
                tekuće <- t'
            k <- k + 1
        inače:
            k <- k + 1

    vrati najbolje

```

Primetimo da protresanje bira **nasumičnu** tačku iz okoline N_k , a ne najbolju. Cilj te odluke jeste

jasna raspodela odgovornosti. Protresanje ima za cilj bežanje od trenutnog lokalnog optimuma, a lokalna pretraga približavanje novom (nadamo se globalnom) optimumu.

Vraćanje na $k = 1$ posle svakog uspeha takođe je bitno. Kada smo pronašli novo, bolje rešenje, pretragu započinjemo upravo iz te tačke.

2.10 Genetski algoritam

Genetski algoritam (Goldberg 1989) jedini je algoritam P-metaheuristika implementiran u radu. P-metaheuristike, za razliku od onih S-tipa, održavaju skup **više rešenja istovremeno**. Kod genetskog algoritma nova rešenja nastaju kombinovanjem postojećih, a selekcija usmerava populaciju ka boljim vrednostima funkcije cilja.

2.10.1 Postupak

ulaz: instanca, budžet B, veličina populacije P, veličina elite E

izlaz: najbolje viđeno rešenje

```
populacija <- P slučajnih permutacija
oceni populaciju i sortiraj je

dok budžet nije potrošen:
  nova <- elita (najboljih E jedinki, nepromenjenih)
  dok nova nije popunjena:
    r1 <- selekcija(populacija)
    r2 <- selekcija(populacija)

    d <- ukrštanje(r1, r2)
    d <- mutacija(d, p_m)

    dodaj (d, dekodiraj(d)) u novu
  populacija <- nova, sortirana
  ažuriraj najbolje viđeno
vрати najbolje
```

2.10.2 Selekcija

Postupak selekcije izvršava se **metodom turnira**. Ona izbor najbolje od slučajnih N jedinki iz trenutne populacije. Veličina podskupa određuje jačinu selekcije, pri tome veća vrednost parametra N češće bira jače jedinke i brže smanjuje raznovrsnost.

2.10.3 Ukrštanje

Prilikom primene genetskog algoritma na JSP nije moguće koristiti *jednopoloziciono ukrštanje*. Razlog za to jeste što spajanje prefiksa jednog i sufiksa drugog roditelja ne garantuje da će izlazni niz biti validan. Recimo, za roditelje (J_1, J_1, J_2, J_2) i (J_2, J_1, J_1, J_2) i ukrštanje na drugoj poziciji dobija se (J_1, J_1, J_1, J_2) , u kome se oznaka J_1 javlja tri puta.

Zbog toga je korišćen algoritam **PPX** (*precedence preservative crossover*), predložen za permutaciona kodiranja u raspoređivanju (Bierwirth, Mattfeld, i Kopfer 1996). Dete se ne dobija sečenjem, već se postepeno gradi koristeći dopuštene elemente:

```

ulaz:  roditelji r1 i r2
izlaz: dete d

a <- kopija(r1)
b <- kopija(r2)
d <- prazan niz

za i od 1 do L:
  izvor <- a ili b, slučajno
  j <- prva oznaka u izvoru
  dodaj j na kraj niza d
  ukloni prvo pojavljivanje oznake j iz a
  ukloni prvo pojavljivanje oznake j iz b
vrati d

```

Uklanjanje iz **oba** roditelja je ključno za ispravnost postupka. Time se garantuje da dete na kraju ima tačno onoliko pojavljivanja svakog posla koliko posao ima operacija. Pored toga, operator čuva i relativan raspored poslova, tj. ako je jedna instanca prethodila drugoj u roditelju, prethodiće i u dobijenom detetu.

2.10.4 Mutacija i elitizam

Nakon ukrštanja, svako dete sa verovatnoćom p_m biva zamenjeno jednom tačkom iz svoje okoline⁵. Bez mutacije algoritam raspolaže samo podacima prisutnim u početnoj populaciji, pa daje lošije rezultate pretrage.

Najbolje jedinke prenose se u narednu generaciju nepromenjene kroz koncept koji nazivamo (*elitizam*). Bez toga najbolje pronađeno rešenje može biti izgubljeno mutacijom ili istisnuto slabijim potomcima.

Tabela 2.8: Parametri genetskog algoritma.

parametar	vrednost
veličina populacije	50
veličina turnira	30% populacije
verovatnoća mutacije p_m	0,3
elitizam	2% populacije

⁵Podsetimo, neposredna okolina tačke podrazumeva sve permutacije trenutnog rasporeda dobijene tačno jednom zamenom.

2.11 Celobrojni linearni model

Kako bismo osigurali nezavisno utvrđivanje optimuma⁶ i proveru ispravnosti dekodera, problem je formulisan i kao zadatak celobrojnog linearnog programiranja. Korišćena je disjunktivna formulacija koju je predložio Alan Manne (Manne 1960).

2.11.1 Promenljive

Tabela 2.9: Promenljive celobrojnog modela.

oznaka	tip	značenje
s_{jk}	realna, ≥ 0	trenutak početka k -te operacije posla j
z_{ab}	binarna	1 ako operacija a prethodi operaciji b na zajedničkoj mašini
C_{max}	realna, ≥ 0	trajanje rasporeda

Binarne promenljive ovde postoje za **svaki par operacija koje se izvršavaju na istoj mašini**

2.11.2 Ograničenja

Redosled unutar posla, za svaku operaciju osim poslednje:

$$s_{j,k+1} \geq s_{jk} + p_{jk}.$$

Isključivost mašine, za svaki par operacija a i b na istoj mašini:

$$s_a + p_a \leq s_b + M(1 - z_{ab}), \quad s_b + p_b \leq s_a + Mz_{ab}.$$

Razmotrimo dva slučaja. Kada je $z_{ab} = 1$, prva nejednakost postaje stvarno ograničenje, dok druga, zbog velike konstante M , prestaje da utiče na rezultat. Analogno važi i u suprotnom slučaju. Ovime izražavamo uslov „ili a pre b , ili b pre a “.

Trajanje rasporeda, za poslednju operaciju svakog posla:

$$C_{max} \geq s_{j,m_j} + p_{j,m_j}.$$

Funkcija cilja je $\min C_{max}$.

Konstanta M mora biti dovoljno velika da ograničenje učini nedelotvornim, a što manja radi kvaliteta linearne relaksacije. Usvojen je zbir svih trajanja u instanci, jer nijedan raspored ne može biti duži od njega.

2.11.3 Veličina modela

Broj promenljivih iznosi $L + m\binom{n}{2} + 1$, a broj ograničenja $n(m-1) + 2m\binom{n}{2} + n$, gde je L ukupan broj operacija:

⁶„Poverenje je dobro, kontrola je još bolja.” – V. I. Lenjin

Tabela 2.10: Veličina celobrojnog modela po instanci.

instanca	promenljivih	ograničenja
mini3	19	27
ft06	127	216
1a01–1a05	276	500
ft10	551	1 000
ft20	1 051	2 000

Model je zapisan pomoću biblioteke PuLP i rešen putem rešavača CPLEX. Rezultati su prikazani u odeljku o egzaktnom rešavanju.

Poglavlje 3

Eksperimentalni rezultati

Kako bismo olakšali kasniju interpretaciju rezultata, za početak ćemo dostaviti kontekstualne informacije o metodologiji usporedbe dve optimizacione metode, veličini eksperimenata i okruženju na kojima su eksperimenti pokretani.

3.1 Okruženje za eksperimentisanje

Sve eksperimentalne skripte izvršavane su na privatnom računaru, čije se najznačajnije karakteristike nalaze u sledećoj tabeli:

Tabela 3.1: Karakteristike računara na kome su izvršena sva merenja.

komponenta	vrednost
procesor	Intel Core Ultra 7 165U
frekvencija	3,71 GHz
broj jezgara	12 fizičkih / 14 logičkih
radna memorija	62,2 GB
operativni sistem	Ubuntu 24.04.4 LTS, 7.0.0-28-generic kernel, x86-64
standardna biblioteka	glibc 2.39
programski jezik	Python 3.12.3, GCC 13.3.0
egzaktni rešavač	IBM ILOG CPLEX 22.2.0.0
biblioteka za modelovanje	PuLP 3.3.2

Vredi napomenuti da su sve razvijene metaheuristike implementirane tako da se izvršavaju na **jednoj niti** bez mogućnosti paralelizacije. Egzaktni rešavač (CPLEX) pokretan je sa podrazumevanim brojem niti, dakle sa svih 14 logičkih jezgara. Zbog te razlike, metode je najpreciznije porediti na osnovu broja poziva funkcije dekodiranja, a ne po utrošenom vremenu. Dužina izvršavanja navedena je kao dodatni, orijentacioni podatak.

3.2 Test instance

Pri evaluaciji razvijenih metoda korišćeno je devet različitih test instanci. Od njih devet, osam je preuzeto iz zbirke OR-Library (Beasley 1990). Instance `ft06`, `ft10` i `ft20` potiču iz rada (Fis-

her i Thompson 1963), dok instance **1a01–1a05** dolaze iz (Lawrence 1984). Poslednja, deveta, instanca **mini3** kreirana je za potrebe ovog rada. Predstavlja relativno malu instancu čiji se optimum može pronaći algoritmom grube sile. Kao takva služila je najpre pri validaciji ispravnosti implementiranih metoda optimizacije. Pregled najvažnijih informacija o svim instancama dat je u narednoj tabeli.

Tabela 3.2: Test instance sa dimenzijama, donjim granicama i objavljenim optimalnim vrednostima.

instanca	$n \times m$	operacija	donja granica	optimum	izvor optimuma
mini3	3×3	9	10	11	iscrpna pretraga
ft06	6×6	36	47	55	(Fisher i Thompson 1963)
1a01	10×5	50	666	666	donja granica
1a02	10×5	50	635	655	(Lawrence 1984)
1a03	10×5	50	588	597	(Lawrence 1984)
1a04	10×5	50	537	590	(Lawrence 1984)
1a05	10×5	50	593	593	donja granica
ft10	10×10	100	655	930	(Fisher i Thompson 1963)
ft20	20×5	100	1119	1165	(Fisher i Thompson 1963)

Donja granica u tabeli 3.2 predstavlja veću od dve trivijalne granice: (1) najdužeg posla i (2) najopterećenije mašine. Način računanja detaljno je opisan u odeljku o donjim granicama. Kod instanci **1a01** i **1a05** ta granica se poklapa sa optimumom, pa svako rešenje koje je dostigne ujedno i **dokazuje** svoju optimalnost, bez potrebe za egzaktnim rešavačem.

Instance su izabrane tako da pokriju što širi opseg težine. Konkretno, **mini3** i **ft06** sve razvijene metode rešavaju do optimuma, pa služe kao provera ispravnosti. Instance **1a01–1a05** razdvajaju metode po pouzdanosti. **ft10** i **ft20** su, uprkos skromnim dimenzijama, poznato teške. Instanca **ft10** je ostala nerešena punih dvadeset šest godina nakon objavljivanja (Jain i Meeran 1999).

3.3 Metodologija

Za početak je potrebno objasniti način izbora veličine eksperimenta. Bilo je potrebno održati balans između validnosti rezultata i razumne upotrebe (pre svega vremenskih) resursa. Sa jedne strane, ne možemo dopustiti da eksperimente izvršavamo na premalom uzorku i time dovedemo u pitanje opštost zaključaka, dok je sa druge strane potrebno da se eksperimenti izvrše u razumnom vremenskom roku.

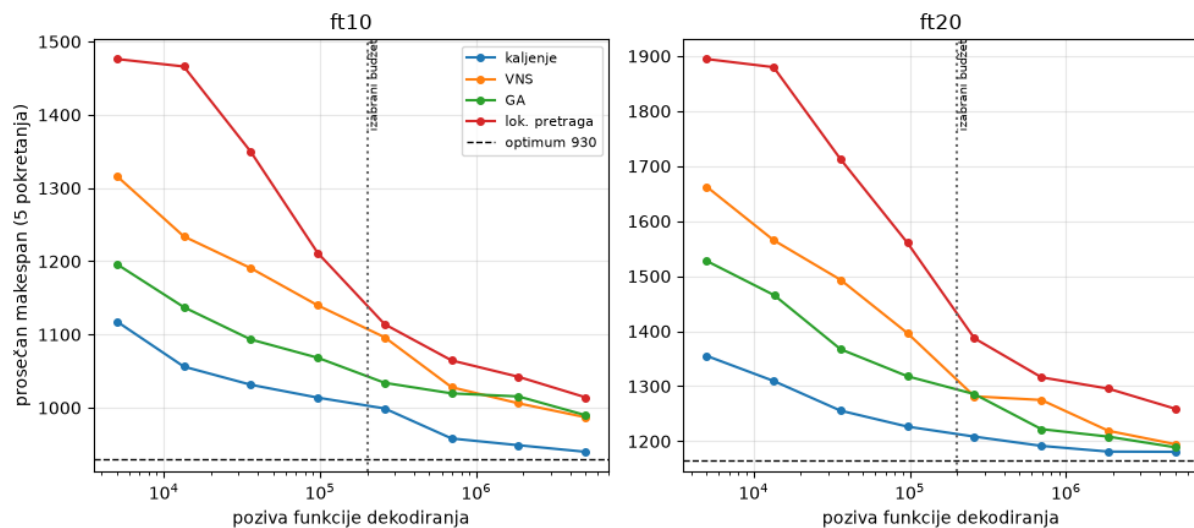
3.3.1 Budžet umesto vremena

Kao što je ranije navedeno, zbog nedostatka konkurentnosti u implementacijama algoritama, efikasnost različitih optimizacionih metoda nije pogodno upoređivati sa egzaktnim rešavačem na osnovu vremena izvršavanja. Kod međusobnog poređenja situacija je nešto bolja, ali se pokazalo

veoma izazovno unapred dozvoliti funkciji određeni vremenski okvir¹, pa je vreme izvršavanja zauzelo ulogu *retrospektivne mere*, dok su sami algoritmi unapred ograničeni brojem poziva funkcije dekodiranja.

Primetićemo ipak, a što će biti detaljnije obrađeno kasnije, da ovaj izbor normalizacije povlači sa sobom pristrasnost prema metodama koje zahtevaju manji broj poziva funkcije cilja po koraku, što im omogućava da dosegnu nešto dalje od metoda koje nemaju tu osobinu.

Svakom algoritmu je unapred data gornja granica broja poziva ove funkcije i taj broj nazivamo *budžetom*. Kao što možemo videti na slici 3.1, prosečan rezultat nastavlja blago da se popravlja i nakon izabrane granice. Međutim, poredak metoda ostaje nepromenjen a preciznije apsolutne vrednost zahtevaju nesrazmerno veći budžet. S tim na umu, izabrani budžet predstavlja kompromis između kvaliteta rešenja i ukupnog trajanje eksperimenta. Primećujemo da se vreme izvršavanja optimizacionog metoda razlikuje i pri istom budžetu, a razlog tome nalazi se u specifičnostima samih algoritama, gde se vreme „rasipa“ na pozive drugih pomoćnih metoda.



Slika 3.1: Prosečna vrednost funkcije cilja u zavisnosti od dodeljenog budžeta, za dve najteže instance. Obe ose prikazane su za pet nezavisnih pokretanja po tački. Uspravna isprekidana linija označava budžet usvojen u ostatku rada.

3.3.2 Broj ponavljanja

Po uzoru na literaturu o metaheuristikama, usvojen je izbor od **30 nezavisnih pokretanja** svake metode nad svakom instancom. Uz devet instanci i četiri različite metode dolazimo do ukupno **1080 pokretanja**. Ovo predstavlja dovoljno veliki uzorak za smisleni prikaz različitih osobina dobijenih rezultata, najpre: prosek, standardnu devijaciju, procenat dostizanja optimuma i slične.

3.3.3 Stohastičnost i reproducibilnost

Svako pokretanje optimizacionog problema u sebi sadrži stohastičnost u vidu **nasumičnog izbora početnog rešenja**. Pored toga, sve četiri metode na stohastičnost nailaze i u drugim

¹Pre svega izazovnost se ogleda u činjenici da isti vremenski okvir na istoj mašini u različitim metodama podrazumeva različit broj koraka.

delovima svoje implementacije, što nas navodi da je potrebno detaljnije obraditi pitanje **reproducibilnosti** rezultata.

Konkretno, postavljanjem iste početne vrednosti za generator pseudonasumičnih brojeva (engl. *seed*) omogućena je potpuna reproducibilnost svih dobijenih rezultata. Ponovno pokretanje priloženog koda dovodi do identičnih rezultata. Takođe *seed* vrednost sačuvana je u priloženim datotekama pa je moguće reprodukovati, ne samo eksperiment, već i jedno zasebno pokretanje optimizacione metode.

3.4 Rezultati testiranja

Dobijeni rezultati prikazani su kroz tri naredne tabele. U prvoj tabeli nalazi se **najbolje pronađeno rešenje** kroz **svih 30 pokretanja**. U drugoj tabeli vidimo prosek i standardnu devijaciju, dok treća tabela nosi informaciju o broju pokretanja u kojima je optimum dostignut.

U prvoj tabeli jasno je izražena informacija o mogućnosti (potencijalu) metode da pronađe najbolje rešenje. Kroz drugu i treću tabelu se ogleda pouzdanost različitih metoda pri višestrukoj upotrebi.

Tabela 3.3: Najbolje pronađeno rešenje kroz 30 pokretanja. Podebljane vrednosti poklapaju se sa objavljenim optimumom.

instanca	optimum	kaljenje	VNS	GA	lok. pretraga
mini3	11	11	11	11	11
ft06	55	55	55	55	55
la01	666	666	666	666	666
la02	655	655	655	655	663
la03	597	597	597	604	597
la04	590	590	590	590	594
la05	593	593	593	593	593
ft10	930	937	971	961	1002
ft20	1165	1173	1192	1180	1249

Tabela 3.4: Prosečna vrednost funkcije cilja kroz 30 pokretanja, sa standardnom devijacijom.

instanca	kaljenje	VNS	GA	lok. pretraga
mini3	11,0 ± 0,0	11,0 ± 0,0	11,0 ± 0,0	11,0 ± 0,0
ft06	55,0 ± 0,0	55,0 ± 0,0	55,0 ± 0,0	55,0 ± 0,0
la01	666,0 ± 0,0	666,0 ± 0,0	670,9 ± 8,9	666,0 ± 0,0
la02	659,0 ± 5,7	660,7 ± 6,6	669,9 ± 9,4	674,5 ± 9,0
la03	606,2 ± 7,4	607,6 ± 7,0	625,6 ± 14,2	623,9 ± 9,9
la04	591,2 ± 2,0	594,1 ± 4,2	606,0 ± 14,2	607,5 ± 7,3
la05	593,0 ± 0,0	593,0 ± 0,0	593,0 ± 0,0	593,0 ± 0,0
ft10	965,2 ± 16,3	1023,8 ± 24,7	1012,1 ± 30,9	1059,6 ± 30,0
ft20	1183,8 ± 11,1	1245,8 ± 29,9	1221,0 ± 26,6	1320,1 ± 40,9

Tabela 3.5: Broj pokretanja, od ukupno 30, u kojima je dostignut poznati optimum.

instanca	kaljenje	VNS	GA	lok. pretraga
mini3	30	30	30	30
ft06	30	30	30	30
1a01	30	30	20	30
1a02	19	16	2	0
1a03	5	3	0	1
1a04	21	11	4	0
1a05	30	30	30	30
ft10	0	0	0	0
ft20	0	0	0	0

Tabela 3.3 pokazuje da instance **mini3**, **ft06** i **1a05** sve tri četiri rešavaju do optimuma. Standardna devijacija na njima je nula (tabela 3.4), pa te instance ne razdvajaju metode i služe isključivo kao provera ispravnosti implementacije.

Poređenje metoda isključivo po najboljem pronađenom rešenju prikriva stvarnu informaciju o upotrebljivosti metode. Na primer, pri rešavanju instance **1a03** sve metode, osim genetskog algoritma, dostižu optimum. Ipak, tabela 3.5 pokazuje da ga lokalna pretraga pronalazi **u samo jednom od trideset pokretanja**, dok simulirano kaljenje to uspeva pet puta. Zbog toga je pri poređenju metoda potrebno uzeti u obzir i njihovo generalno ponašanje. Nepouzdanost je ograničiti se samo na najbolje primere.

Na instancama **1a02**, **1a03** i **1a04** razlike su najizraženije. Kaljenje dostiže optimum redom 19, 5 i 21 put, promenljive okoline 16, 3 i 11 puta, dok lokalna pretraga ne uspeva gotovo nikada.

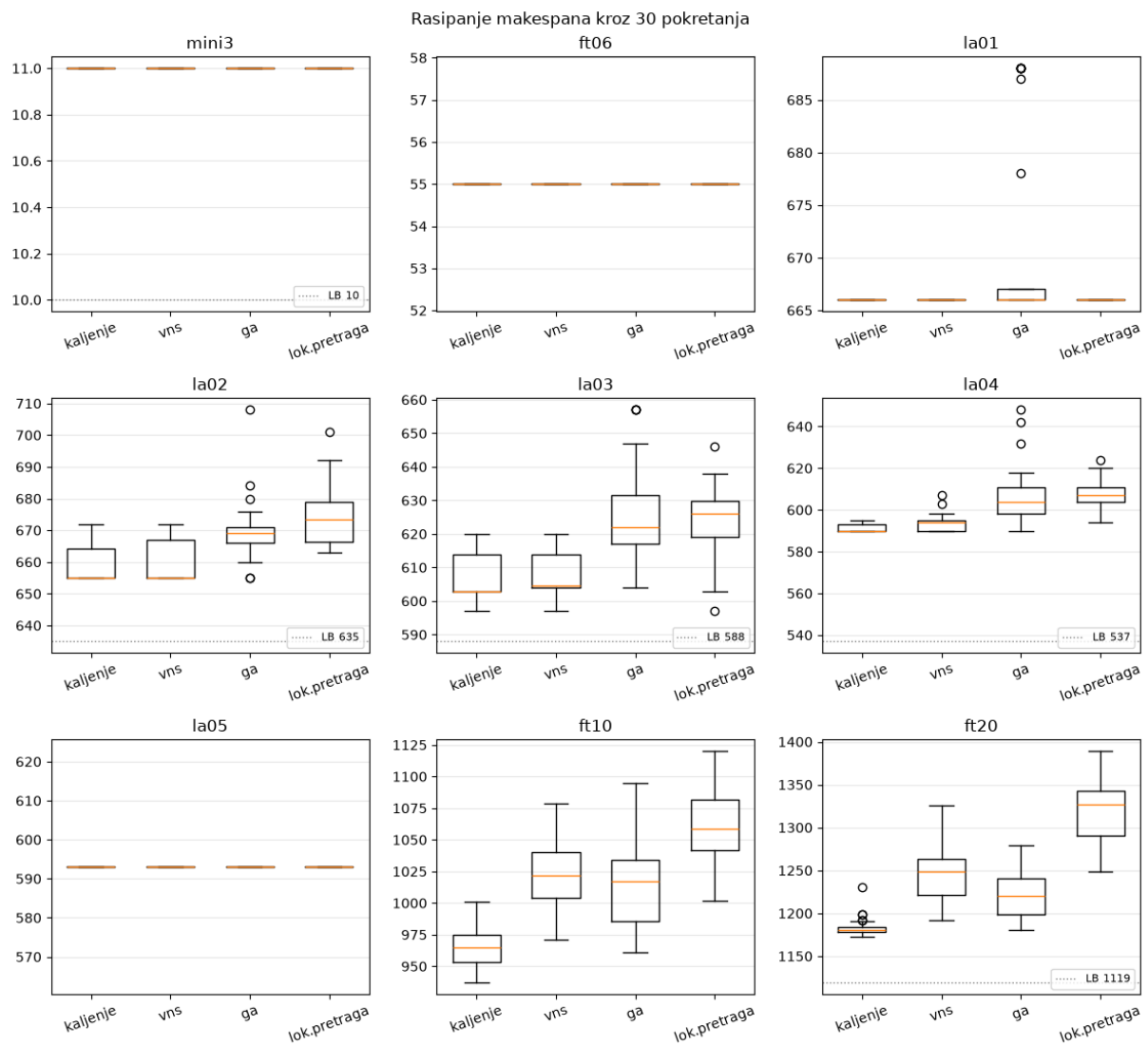
Instanca **1a03** se u rezultatima izdvaja. Naime, iako je istih dimenzija kao **1a02** i **1a04**, sve metode je rešavaju znatno teže. Uzrok nije utvrđen u okviru ovog rada.

Na najvećim instancama, **ft10** i **ft20**, nijedna metoda ne dostiže objavljeni optimum. Ipak, simulirano kaljenje je i tu najbliže, sa prosečnim odstupanjem od 3,8% odnosno 1,6%.

Posmatrano na svim instancama, prosečno odstupanje od optimuma iznosi 0,86% za kaljenje, 2,26% za promenljive okoline, 2,68% za genetski algoritam i 4,19% za lokalnu pretragu sa ponovnim pokretanjem. Poredak je isti u sve tri tabele, što ga čini pouzdanim zaključkom.

3.5 Egzaktno rešavanje

Radi provere ispravnosti razvijenih metoda i utvrđivanja stvarnih optimuma, problem je rešen i egzaktno, celobrojn timer linearnim programiranjem. Korišćena je disjunktivna formulacija (Mann 1960), opisana u odeljku o celobrojn timer modelu, dok je model rešen putem rešavača CPLEX. Pritom, svakoo pokretanje vremenski je unapred ograničeno na jedan sat i zahtevana je stroga optimalnost. Rezultati egzaktnog rešavanja dati su u sledećoj tabeli:



Slika 3.2: Rasipanje vrednosti funkcije cilja kroz 30 pokretanja, po instanci i metodi. Kutija obuhvata srednjih 50% rezultata, linija u njoj je medijana, a kružići označavaju odudarajuće vrednosti (*engl. outliers*).

Tabela 3.6: Rezultati egzaktnog rešavanja. Podebljane vrednosti su dokazano optimalne.

instanca	promenljivih	ograničenja	rešenje	donja granica	odstupanje	vreme [s]
mini3	19	27	11	11	0	0,1
ft06	127	216	55	55	0	0,3
la01	276	500	666	666	0	5,2
la02	276	500	655	655	0	3,0
la03	276	500	597	597	0	2,7
la04	276	500	590	590	0	1,6
la05	276	500	593	593	0	9,8
ft10	551	1000	930	930	0	50,8
ft20	1051	2000	1182	657	44,4 %	> 3600

Osam od devet instanci rešeno je do dokazane optimalnosti i sve dobijene vrednosti poklapaju se sa objavljenim optimumima iz tabele 3.2. Ovime je nezavisno potvrđeno da implementirani dekodier ne proizvodi neizvodljive rasporede.

Primetimo da vreme rešavanja raste izrazito brzo sa veličinom instance: 0,3 sekunde za **ft06**, oko 3 sekunde za instance **la01–la05**, 50,8 sekundi za **ft10**, dok **ft20** nije rešen ni nakon sat vremena. Upravo ovde vidimo najveću vrednost metaheuristika.

Posvetimo na kratko pažnju instanci **ft20**. Nakon sat vremena rešavač je pronašao rešenje vrednosti 1182 i dokazao donju granicu 657. U pitanju je razilaženje od preko 44%. Poređenja radi, simulirano kaljenje je u istom okruženju pronašlo rešenje vrednosti 1173 za oko pet sekundi. Rezultat je da je primenom metode optimizacije putem metaheuristika pronađeno **bolje rešenje u vremenu manjem za tri reda veličine**.

Pored toga, jednostavna donja granica opisana u odeljku o donjim granicama iznosi za **ft20** **1119**, što je znatno jače ograničenje od granice 657 koju je rešavač dokazao za sat vremena. Kombinovanjem te granice sa najboljim pronađenim rešenjem dobija se interval [1119, 1173], odnosno raskol od 4,6% umesto 44,37%.

3.6 Poređenje sa rezultatima iz literature

U cilju procene realne upotrebljivosti razvijenih metoda, njihovi rezultati upoređeni su sa optimalnim vrednostima objavljenim u literaturi. Za instance iz zbirke OR-Library te vrednosti su odavno poznate i dosegnute su specijalizovanim metodama, između ostalih egzaktnim algoritmi-
ma granjanja i ograđivanja, i tabu pretragom prilagođenom strukturi problema.

Na sedam od devet instanci simulirano kaljenje dostiže objavljeni optimum. Na preostale dve, **ft10** i **ft20**, odstupanje iznosi 0,75% odnosno 0,69%.

Tabela 3.7: Poređenje najboljih pronađenih rešenja sa objavljenim optimumima. Podebljane vrednosti poklapaju se sa optimumom.

instanca	optimum	najbolje (kaljenje)	odstupanje	ILP	odstupanje
mini3	11	11	0	11	0
ft06	55	55	0	55	0

instanca	optimum	najbolje (kaljenje)	odstupanje	ILP	odstupanje
1a01	666	666	0	666	0
1a02	655	655	0	655	0
1a03	597	597	0	597	0
1a04	590	590	0	590	0
1a05	593	593	0	593	0
ft10	930	937	0,75 %	930	0
ft20	1165	1173	0,69 %	1182	1,46 %

Na osnovu priloženih rezultata jasno se vidi da razvijene metode ipak **ne nadmašuju rezultate iz literature**. Objavljene vrednosti za instance **ft10** i **ft20** dostignute su metodima koji pri optimizaciji koriste strukturu samog problema (Nowicki i Smutnicki 1996), dok su naše implementirane metode opšte prirode i oslanjaju se na pronalaženje jednostavne okoline. Ovakav ishod je očekivan i u skladu sa obimom rada.

Vredi, međutim, još jednom osvrnuti se na odnos utrošenog vremena. Rešenje vrednosti 1173 za instancu **ft20** simulirano kaljenje pronalazi za oko pet sekundi, dok egzaktni rešavač ni nakon sat vremena ne uspeva da pronađe bolje od 1182. Za praktičnu primenu, u kojoj se raspored često mora dobiti u kratkom roku, odstupanje ispod jednog procenta uz vreme izvršavanja od nekoliko sekundi predstavlja upotrebljiv rezultat.

3.7 Diskusija

Prilikom razvoja i merenja uočeno je nekoliko pojava koje se ne vide iz zbirnih tabela, a koje su bitno uticale na konačan izbor metoda i njihovih parametara.

3.7.1 Uža okolina ne donosi bolji rezultat

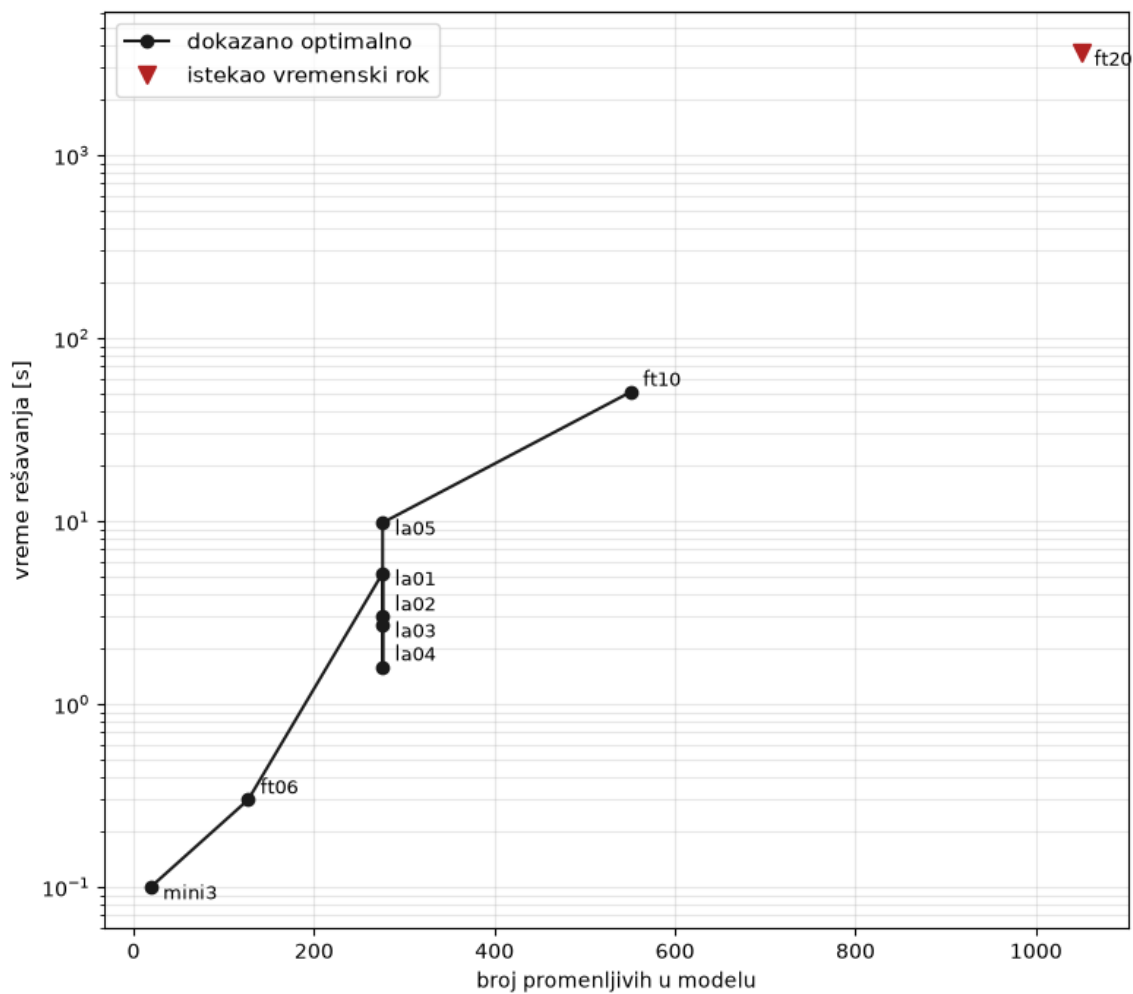
Okolina zasnovana na kritičnom putu, u literaturi poznata kao N_1 (Laarhoven, Aarts, i Lenstra 1992), razmatra samo zamene susednih operacija na kritičnom putu koje se izvršavaju na istoj mašini. Time se broj kandidata po koraku drastično smanjuje. Na primer, sa 540 na 6,7² kod instance **ft06**, sa 4500 na 17,5 kod **ft10** i sa 4750 na 31,6 kod **ft20**, dakle između 81 i 259 puta.³

Uprkos tome, pri izjednačenom broju poziva funkcije dekodiranja ta okolina daje **lošiji** rezultat od proste zamene dva elementa. Kod višestruko pokrenute lokalne pretrage na instanci **ft10** prosek je 1063,6 naspram 1050,5, a kod simuliranog kaljenja 1001,0 naspram 956,7. Na instanci **ft20** razlika je najizraženija: 1223,7 naspram 1182,1.

Postoje dva glavna uzroka zbog čega se pojavljuju ovakvi rezultati. Prvi razlog nalazi se upravo u dizajnu samog eksperimenta, odnosno u odluci da upoređivanje metoda normalizujemo na osnovu budžeta. Naime, iako se pokazalo efektno pri usporedbi metoda optimizacije u njihovom osnovnom obliku, pri pronalaženju N_1 poziva se funkcija dekodiranja, što samo po sebi umanjuje budžet. Drugo, uža okolina daje plići pojam lokalnog optimuma, što navodi pretragu na ranije zaustavljanje i to na lošijem mestu. Kod simuliranog kaljenja postoji i treći razlog. Naime,

²Date su prosečne vrednosti broja kandidata

³Ova merenja sprovedena su naknadno, s obzirom na neobećavajuće rezultate



Slika 3.3: Vreme rešavanja celobrojnog modela u zavisnosti od broja promenljivih. Vertikalna osa je logaritamska. Instanca **ft20** nije rešena u zadatom roku od sat vremena, pa njena vrednost predstavlja donju procenu potrebnog vremena.

ograničavanjem izbora na kritični put oduzimaju se neutralni potezi, koji ne menjaju vrednost funkcije cilja odmah, ali otvaraju put kasnijim poboljšanjima.

U literaturi N_1 i srodne okoline daju vrlo dobre rezultate, ali u sprezi sa tabu listom i inkrementalnom procenom vrednosti (Nowicki i Smutnicki 1996), čime se oba navedena nedostatka uklanjaju. Bez tih dopuna, sama okolina ispostavlja se nedovoljnom.

3.7.2 Složenija metoda nije nužno bolja

Metoda promenljivih okolina koristi lokalnu pretragu kao potprogram i nad njom gradi mehanizam izlaska iz lokalnog optimuma. Očekivano bi bilo da nadmaši simulirano kaljenje, koje je konceptualno jednostavnije. Merenja pokazuju suprotno: pri istom budžetu prosečno odstupanje iznosi 2,26% za promenljive okoline naspram 0,86% za kaljenje.

Razlog je u ceni jednog koraka. Jedan silazak lokalne pretrage na instanci `ft10` troši oko 39 000 poziva funkcije cilja, pa pri budžetu od 200 000 poziva metoda stigne da izvede svega nekoliko silazaka. Simulirano kaljenje u istom budžetu razmotri 200 000 pojedinačnih poteza. Pri ovako postavljenom poređenju prednost ima metoda sa jeftinijim korakom.

3.7.3 Odnos prema egzaktnom rešavanju

Rezultati na instanci `ft20` pokazuju granicu primenljivosti egzaktnog pristupa. Rešavač je za sat vremena pronašao rešenje vrednosti 1182 i dokazao donju granicu 657, dok je simulirano kaljenje za oko pet sekundi pronašlo rešenje vrednosti 1173. Uz to je jednostavna donja granica iz odeljka o donjim granicama, izračunata u zanemarljivom vremenu, iznosila 1119, što je znatno više od granice koju je rešavač dokazao.

Kombinovanjem sopstvenih rezultata, dakle donje granice 1119 i najboljeg pronađenog rešenja 1173, dobija se procena optimuma sa ostupanjem od 4,6%, umesto 44,37% koliko je iznosilo odstupanje egzaktnog rešavača.

Poglavlje 4

Zaključak

Poglavlje 5

Literatura

- Applegate, David, i William Cook. 1991. „A Computational Study of the Job-Shop Scheduling Problem“. *ORSA Journal on Computing* 3 (2): 149–56.
- Beasley, J. E. 1990. „OR-Library: Distributing Test Problems by Electronic Mail“. *Journal of the Operational Research Society* 41 (11): 1069–72.
- Bierwirth, Christian. 1995. „A Generalized Permutation Approach to Job Shop Scheduling with Genetic Algorithms“. *OR Spektrum* 17 (2–3): 87–92.
- Bierwirth, Christian, Dirk C. Mattfeld, i Herbert Kopfer. 1996. „On Permutation Representations for Scheduling Problems“. U *Parallel Problem Solving from Nature — PPSN IV*, 1141:310–18. Lecture Notes in Computer Science. Springer.
- Carlier, J., i E. Pinson. 1989. „An Algorithm for Solving the Job-Shop Problem“. *Management Science* 35 (2): 164–76.
- Fisher, H., i G. L. Thompson. 1963. „Probabilistic Learning Combinations of Local Job-Shop Scheduling Rules“. U *Industrial Scheduling*, uredio J. F. Muth i G. L. Thompson, 225–51. Englewood Cliffs, New Jersey: Prentice Hall.
- Garey, M. R., D. S. Johnson, i Ravi Sethi. 1976. „The Complexity of Flowshop and Jobshop Scheduling“. *Mathematics of Operations Research* 1 (2): 117–29.
- Giffler, B., i G. L. Thompson. 1960. „Algorithms for Solving Production-Scheduling Problems“. *Operations Research* 8 (4): 487–503.
- Goldberg, David E. 1989. *Genetic Algorithms in Search, Optimization and Machine Learning*. Reading, Massachusetts: Addison-Wesley.
- Jain, A. S., i S. Meeran. 1999. „Deterministic Job-Shop Scheduling: Past, Present and Future“. *European Journal of Operational Research* 113 (2): 390–434.
- Janičić, Predrag, i Mladen Nikolić. 2025. *Veštačka inteligencija*. 5. izd. Beograd: Univerzitet u Beogradu, Matematički fakultet.
- Kapunac, Stefan. 2026. „Računarska inteligencija — materijali sa vežbi“. Matematički fakultet, Univerzitet u Beogradu.
- Kirkpatrick, S., C. D. Gelatt, i M. P. Vecchi. 1983. „Optimization by Simulated Annealing“. *Science* 220 (4598): 671–80.
- Laarhoven, Peter J. M. van, Emile H. L. Aarts, i Jan Karel Lenstra. 1992. „Job Shop Scheduling by Simulated Annealing“. *Operations Research* 40 (1): 113–25.
- Lawrence, S. 1984. „Resource Constrained Project Scheduling: An Experimental Investigation of Heuristic Scheduling Techniques (Supplement)“. Pittsburgh, Pennsylvania: Graduate School

- of Industrial Administration, Carnegie-Mellon University.
- Manne, Alan S. 1960. „On the Job-Shop Scheduling Problem“. *Operations Research* 8 (2): 219–23.
- Metropolis, Nicholas, Arianna W. Rosenbluth, Marshall N. Rosenbluth, Augusta H. Teller, i Edward Teller. 1953. „Equation of State Calculations by Fast Computing Machines“. *The Journal of Chemical Physics* 21 (6): 1087–92.
- Mladenović, N., i P. Hansen. 1997. „Variable Neighborhood Search“. *Computers & Operations Research* 24 (11): 1097–1100.
- Nowicki, Eugeniusz, i Czesław Smutnicki. 1996. „A Fast Taboo Search Algorithm for the Job Shop Problem“. *Management Science* 42 (6): 797–813.
- Wagner, Harvey M. 1959. „An Integer Linear-Programming Model for Machine Scheduling“. *Naval Research Logistics Quarterly* 6 (2): 131–40.